Tecsup VIRTUAL

BOOTCAMP FRONTEND & REACT · BLOQUE REACT · CLASE 8

El proyecto final arranca

Cómo se califica, cómo se elige el tema, y la base de un proyecto nuevo creada en vivo desde cero: arquitectura en capas, API, sesión, rutas y producción.

L·M·V Última sesión del bloque

MASTERCLASS

ÍNDICE

ÚLTIMA SESIÓN DEL BLOQUE

El plan de hoy

0. **Reapertura**
el punto de corte + dos tiendas en producción

A. **Remate de la clase 7**
solo lo pendiente · guion de la clase 7

B. **La rúbrica, criterio por criterio**
20 puntos · 12 temas · bonus · sustentación

C. **Del tema al contrato**
tres preguntas antes de codear · APIs públicas

D. **La base del proyecto, desde cero**
un recetario: capas · API · sesión · rutas · deploy

E. **Integración con backend propio**
Spring Boot · CORS · proxy de Vite · el adaptador

F–G. **Errores frecuentes + cierre del bloque**
plan hasta la sustentación · tareas = hitos

Receso de 15 min · verde = **el corazón del día** · cierre a las 3:45

RÚBRICA

ESCALA VIGESIMAL · RÚBRICA PUBLICADA EN EL REPOSITORIO DEL CURSO

Cómo se **califica**

3

Proceso

avance en el repo después de cada clase: 2 réplica + tareas · 1 solo réplica · 0 sin entrega. Se promedia y escala.

11

Proyecto desplegado

los 12 temas del temario, cada uno Logrado / Parcial / Insuficiente / Ausente. Bonus hasta +1 (tope 11).

3

Sustentación

10 min de demostración en producción + defensa de una decisión propia. 5 min de preguntas.

3

Dominio conceptual

2 o 3 preguntas al azar del banco, respondidas sobre TU código.

Requisitos de recepción (sin puntos; sin ellos no se corrige): `npm run build` sin errores · aplicación desplegada en Vercel y operativa desde cualquier dispositivo · consola del navegador sin errores en el flujo principal.

RÚBRICA

11 PUNTOS · LOGRADO 100% · PARCIAL 60% · INSUFICIENTE 20% · AUSENTE 0

Los 12 temas del proyecto

Tema	Pts	Logrado	Tema	Pts	Logrado
1 · Vite + JSX	0.50	proyecto propio que compila, JSX correcto	7 · React Router	1.00	detalle por URL, 404 propia, rutas operativas tras recargar
2 · Componentes + Props	1.00	uno por archivo, props tipadas, interfaces del dominio	8 · Formularios	1.00	controlado, validación por campo, labels, envío bloqueado
3 · Listas y condicionales	0.75	key = id, sin el "0" visible, responsive, semántica	9 · Hooks + Context	1.00	la colección en Context, un hook propio con uso real
4 · Eventos + useState	1.00	estado mínimo, derivados, inmutabilidad	10 · JWT + sesión + .env	1.50	token usado (Bearer), expira, rutas protegidas que retornan
5 · useEffect	1.00	dependencias, limpieza, persistencia con validación	11 · Optimización	0.25	memo o lazy con criterio y evidencia
6 · API + carga + errores	1.75	tres estados, reintentar, AbortController, sin catch vacío	12 · Vercel	0.25	operativa, rewrite de SPA

Cinco requisitos mínimos, sea cual sea el tema: entidad principal desde una API real · colección del usuario con persistencia · formulario principal con validación · sesión con áreas protegidas · desplegada. **TypeScript acotado:** interfaces del dominio usadas + props tipadas.

TRES PREGUNTAS ANTES DE ABRIR VS CODE

Del tema al contrato

1 · ¿Qué entidad?

UNA sola: recetas, películas, libros, eventos, pokémon, ejercicios. Cinco o seis campos que de verdad vas a mostrar, en español.

2 · ¿De qué API?

Real, gratuita, JSON: DummyJSON (recipes, posts, users, todos), TMDB (películas), PokeAPI, Open Library, países, clima.

3 · ¿Qué colección del usuario?

Favoritos, "por cocinar", lista de lectura, reservas, "ya visto". Es tu carrito: Context + persistencia + su formulario.

```
// el ejemplo de hoy: recetas de dummyjson.com/recipes
export interface Receta { id: number; nombre: string; imagen: string; cocina: string;
dificultad: string; minutos: number; ingredientes: string[]; }
// mapa de rutas: / · /receta/:id · /perfil (protegida) · /login · *
```

Con el **contrato** y el **mapa de rutas** en papel, el resto es el mismo orden de la tienda. La API se traduce en UN archivo: el adaptador.

LO QUE SE CONSTRUYE HOY · LO QUE TE TOCA A TI

La base del proyecto

```
src/
  dominio/          tipos.ts ← tu contrato
  infraestructura/  api.ts (adaptador) · datos.ts · auth.ts · almacen.ts
  aplicacion/       SesionContext.tsx · useSesion.ts ← copiados de la tienda
                    + ColeccionContext.tsx · useColeccion.ts ← TAREA
  ui/components/    Header · Footer · TarjetaReceta · RutaProtegida
  ui/pages/         Home · Detalle · Perfil · Login · NoEncontrada + el formulario ← TAREA
  ui/layout/        Layout.tsx
  App.tsx · main.tsx · vite-env.d.ts    raíz: .env · .env.example · vercel.json
```

Hoy sale con: capas, API con adaptador y tres estados, detalle por URL, sesión real que expira, ruta protegida, lazy, deploy.

Te falta (las tareas): la colección en Context,

BACKEND

ÚLTIMO TEMA DEL TEMARIO · DEL FRONTEND HACIA SPRING BOOT

Integración con un backend propio

No cambia NADA de React

componentes · props · estado · derivados · efectos · rutas · Context · hooks propios · los tres estados de carga · RutaProtegida · memo · lazy

Cero líneas, porque la API está detrás de un adaptador.

Cambian 4 cosas, todas en infraestructura

1 · la URL base → variable de entorno
2 · la forma del JSON → el adaptador (Spring pagina en `content`)
3 · el campo del token (`token`, no `accessToken`) y la ruta de login
4 · los códigos: `401` sin token o vencido · `403` sin permiso

Cambiar de backend = 1 archivo por asunto + 1 línea de import.

```
// CORS: el navegador bloquea, no el backend. La respuesta llegó; no te la entrega.
blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present
// producción: que el backend autorice tu dominio .vercel.app (y sea https)
// desarrollo: proxy de Vite → server: { proxy: { '/api': { target: 'http://localhost:8080',
changeOrigin: true } } }
```

PÍDELO POR ESCRITO ANTES DE ESCRIBIR EL ADAPTADOR

El contrato que le pides al backend

1 · URL base	en desarrollo y en producción. La de producción, **con https**: una página HTTPS no puede pedir a HTTP.
2 · Rutas	las que vas a usar, con su método: `GET /api/recetas`, `GET /api/recetas/{id}`, `POST /api/auth/login`…
3 · Ejemplos	una respuesta **real** de cada ruta, pegada tal cual. Con eso escribes la interfaz cruda sin inventar nombres.
4 · Token	qué ruta lo entrega, qué campos recibe, y **cómo se llama** el campo que vuelve.
5 · Duración	cuánto vale el token, y en qué unidad viene (segundos o milisegundos).
6 · Errores	qué códigos devuelven (401 · 403 · 404 · 500) y si mandan un mensaje en el cuerpo que puedas mostrarle al usuario.
7 · CORS	si está habilitado para `localhost:5173` y para tu dominio de Vercel.

La mitad de los problemas de integración se resuelven con **un ejemplo de respuesta pegado en el chat**.

TODOS BAJAN PUNTOS QUE YA TENÍAS GANADOS

Errores frecuentes en proyectos finales

1 · la tienda con otro nombre

se nota en la primera pregunta de la sustentación. Tu contrato, tu API, tu colección

2 · todo en App

componentes de 300 líneas. La rúbrica pide un componente por archivo con una responsabilidad

3 · sin carga ni errores

pantalla en blanco mientras carga y un catch vacío. Es el tema que más pesa: 1.75

4 · token guardado, nunca enviado

ninguna petición con Authorization: Bearer. La rúbrica lo pide explícitamente

5 · un commit gigante la noche anterior

el proceso vale 3 puntos y el historial de commits es la evidencia

6 · 404 al recargar o .env en el repo

falta vercel.json; o una clave publicada. Revisar antes de entregar

CIERRE

ENSÁYALA CON RELOJ

Sustentación: 10 + 5

min 1	**Qué es y para quién**, en dos frases.
min 2–6	**El flujo completo en producción:** listado · búsqueda · detalle · agregar a tu colección · cerrar sesión · intentar la ruta protegida · login · el formulario fallando una validación y luego pasando.
min 7–10	**UNA decisión de diseño tuya**, con razones: por qué esa API, por qué la colección vive en Context, por qué separaste tal archivo.
+5 min	**Preguntas sobre tu código** (banco conceptual) y, quizá, un cambio pequeño en vivo: agregar un campo al contrato y propagarlo.

Preguntas de muestra: "Señala un dato derivado en tu App: ¿por qué no es un useState?" · "Muestra la función de limpieza de un efecto: ¿qué pasa si la borras?" · "Pon la aplicación en Offline: ¿qué ve el usuario y por qué eso es correcto?" · "¿Qué pasaría si tu listado usara el índice como key?"

CIERRE

UN COMMIT AL TERMINAR CADA HITO, NO AL FINAL

Plan de trabajo

Hito 1	**La base con TU tema:** contrato, adaptador, listado con estados, detalle, sesión, deploy. Es replicar lo de hoy.
Hito 2	**La colección del usuario:** Context + hook propio, botón en la tarjeta, listado en la página protegida, persistencia.
Hito 3	**El formulario principal:** controlado, validación por campo, labels, guarda algo real.
Hito 4	**Pulir:** skeleton, 404 propia, título por página, README con la URL y la sección "Decisión de diseño".
Hito 5	**Ensayar la sustentación** con reloj.

Entrega: [FECHA] · **Sustentaciones:** [FECHA] · Reviso los repos entre medio: si te trabas, un commit con el error y un mensaje alcanzan.

TAREAS

TAREAS = HITOS DEL PROYECTO · REPO PROPIO, BUILD EN CERO, URL EN EL README

5 tareas

FÁCIL	**La base con tu tema**: replica lo de hoy con TU contrato y TU API. Desplegada, con la URL en el README.
INTERMEDIO	**La colección del usuario**: Context + hook propio calcados del carrito, botón en la tarjeta, listado en la página protegida, persistencia con comprobación al leer.
INTERMEDIO	**El formulario principal**: controlado, validación por campo con mensajes junto al input, labels, envío bloqueado con errores. Guarda algo real.
DIFÍCIL	**El README que sustenta**: sección "Arquitectura" (propósito de cada carpeta con un ejemplo) y sección "Decisión de diseño". Es tu guion de la sustentación.
DIFÍCIL	**El bonus completo**: una interfaz propia para el adaptador (`RepositorioRecetas`), `api.ts` la implementa y la aplicación depende de la interfaz. Puertos y adaptadores de verdad.

Cierre del bloque React

Lo que construiste en ocho clases

```
componentes · props · estado · derivados · efectos · limpieza
API con adaptador · estados de carga · rutas · formularios · Context · hooks propios
sesión con JWT · rutas protegidas · arquitectura en capas · memo/lazy · producción
integración con un backend propio · CORS · proxy de desarrollo
// una tienda en internet, y hoy otro proyecto desde cero en una tarde
```

Eso es lo que lleva un desarrollador frontend junior a su primera entrevista — y la URL para probarlo. Lo que sigue es tu proyecto: mío es revisar, responder y calificar; tuyo es construir.

Nos vemos en las sustentaciones. Gracias por estas ocho clases.